écentric — Complete Application Context for Claude Code

> **Purpose**: This document is the single source of truth for building everything that happens after a user clicks "Get Started" on the écentric landing page. It covers authentication, onboarding, the full dashboard application, data models, API contracts, UI/UX behavior, edge cases, error states, and feature specifications down to the component level.

Table of Contents

1. Product Summary
2. Architecture & Tech Stack
3. Authentication & Account Creation
4. Onboarding Flow
5. Dashboard — Global Layout & Navigation
6. Dashboard — Home / Overview
7. Product Catalog
8. Product Detail View
9. Competitor Intelligence
10. Optimization Center
11. UCP & Structured Data
12. Model Drift & Alerts
13. Reports & Exports
14. Settings
15. Billing & Plan Management
16. Notifications System
17. Data Model — Complete Schema
18. API Routes & Contracts
19. Simulation Engine — Internal Specification
20. Auto-Optimization Engine
21. Tier Gating & Feature Flags
22. Empty States, Loading States & Error Handling
23. Accessibility & Responsive Design
24. Analytics & Tracking Events

25. Product Hunt Launch Considerations

---

1. Product Summary

écentric is the autonomous optimization engine for agent commerce. It helps e-commerce merchants win when AI shopping agents (GPT, Claude, Gemini, Perplexity, and others) make purchasing decisions on behalf of consumers.

Core loop: Simulate → Diagnose → Optimize → Publish → Repeat.

The insight: AI agents don't see ads, scroll pages, or respond to branding. They read structured data, compare specs, and choose in milliseconds. If your product data is incomplete or poorly structured, agents skip you — and you never know it happened.

Pricing tiers:

Tier	Price	Product Limit	Core Capabilities
Scan (Free)	$0	10 products	One-time simulation across 3 models, basic attribute gap report
Compete	$149/mo	100 products	Daily simulations, competitor battlecards, manual optimization suggestions
Dominate	$449/mo	1,000 products	Continuous simulations, auto-optimization + one-click publish, model drift alerts, UCP generation
Enterprise	Custom	Custom	SSO/SAML, API access, dedicated simulation environments, custom reporting

---

2. Architecture & Tech Stack

Layer	Technology	Notes
Frontend	Next.js 14+ (App Router) + TypeScript + Tailwind CSS + shadcn/ui	Single repo, server components by default
Backend API	Next.js API routes (Route Handlers)	Server actions for mutations where appropriate
Database	PostgreSQL via Supabase (or Neon)	Row-level security, real-time subscriptions for live simulation updates

Layer	Technology	Notes
Auth	Clerk	OAuth (Google, Shopify), email+password, MFA support, organization/team management
AI Simulation	Server-side calls to OpenAI, Anthropic, Google Generative AI APIs	Structured prompts with JSON output mode
Job Queue	BullMQ + Redis (Upstash or Railway-hosted)	Async simulation jobs, scheduled daily/weekly re-runs
E-commerce Integrations	Shopify Admin API (GraphQL), WooCommerce REST API, Amazon SP-API, CSV/XML import	OAuth flows for platform connections
Payments	Stripe	Subscription billing, usage metering for Enterprise, Shopify 20% rev-share handling
File Storage	Supabase Storage or S3	CSV uploads, product images, export files
Hosting	Vercel (frontend + API) + Railway or Fly.io (workers)	Edge functions for low-latency API responses
Monitoring	PostHog (product analytics), Sentry (error tracking), Axiom (logs)	
Email	Resend	Transactional emails, notification digests, onboarding drip sequences
Real-time	Supabase Realtime or Pusher	Live simulation progress updates, dashboard refresh

Project Structure

```
/app
  /(auth)
    /signup
    /login
    /forgot-password
  /(onboarding)
    /connect
    /select-products
    /competitors
    /first-scan
  /(dashboard)
    /layout.tsx              → Sidebar + topbar shell
```

```
    /page.tsx              → Dashboard home/overview
    /products
      /page.tsx            → Product catalog table
      /[id]/page.tsx       → Product detail view
    /competitors
      /page.tsx            → Competitor intelligence hub
    /optimize
      /page.tsx            → Optimization center / queue
    /ucp
      /page.tsx            → UCP & structured data manager
    /reports
      /page.tsx            → Reports & exports
    /settings
      /page.tsx            → Settings hub
      /team
      /integrations
      /notifications
      /billing
      /simulation-config
  /components
    /ui                    → shadcn/ui primitives
    /dashboard             → Dashboard-specific components
    /onboarding            → Onboarding step components
    /shared                → Shared components (charts, tables, modals)
  /lib
    /api                   → API client functions
    /simulation            → Simulation engine logic
    /integrations          → Shopify, WooCommerce, etc.
    /stripe                → Billing utilities
    /utils                 → Helpers, formatters, constants
  /server
    /actions               → Server actions
    /jobs                  → BullMQ job processors
    /services              → Business logic services
```

---

3. Authentication & Account Creation

Route: `/signup`

Entry point: User clicks "Get Started" or "Scan Your Store Free" on the landing page.

Sign-Up Form Fields

Field	Type	Required	Validation
Email	email input	Yes	Valid email format, not already registered
Password	password input	Yes	Min 8 chars, 1 uppercase, 1 number
Full Name	text input	Yes	Min 2 chars
Company / Brand Name	text input	Yes	Min 2 chars
Role	select dropdown	Yes	Options: Founder / CEO, Marketing Lead, E-commerce Manager, Agency, Developer, Other
How did you hear about us?	select dropdown	No	Options: Product Hunt, Twitter/X, LinkedIn, Google Search, Shopify App Store, Friend/Colleague, Newsletter, Podcast, Other

OAuth Options

- **Continue with Google** — Standard Google OAuth
- **Continue with Shopify** — Shopify OAuth (pre-fills company name, auto-connects store on completion)
- Shopify OAuth is the preferred path — it skips the "Connect Your Store" step entirely

Post-Signup Behavior

1. Create user record in `users` table
2. Create organization record in `organizations` table (user is owner)
3. Set `onboarding_status = 'pending'` on organization
4. If signed up via Shopify OAuth → auto-connect store, set `onboarding_step = 'select-products'`
5. If signed up via email/Google → set `onboarding_step = 'connect'`
6. Redirect to appropriate onboarding step
7. Fire `user_signed_up` analytics event
8. Trigger welcome email via Resend (delay 2 min)

Route: `/login`

- Email + password login
- OAuth login (Google, Shopify)
- "Forgot password" link → `/forgot-password`

- If user has `onboarding_status = 'pending'`, redirect to their current onboarding step
- If user has `onboarding_status = 'complete'`, redirect to `/dashboard`

---

4. Onboarding Flow

The onboarding is a linear, 4-step wizard with a persistent progress bar at the top. Each step has a dedicated route. Users can navigate back to previous steps but cannot skip ahead. The goal is to deliver the "aha moment" — a visible win rate score — within 2 minutes of signing up.

Global Onboarding UI

- **Progress bar**: 4 steps, labeled: Connect Store → Select Products → Add Competitors → Run Scan
- **Step indicator**: Current step is highlighted (blue), completed steps show checkmark, future steps are grayed
- **"Skip for now"**: Available on Steps 2 and 3 (uses smart defaults)
- **Persistent sidebar** (desktop): Shows a motivational message that updates per step
  - Step 1: "Let's connect your catalog so agents can find your products."
  - Step 2: "Pick the products you want to test against AI agents."
  - Step 3: "Who are you competing against? We'll find out who's winning."
  - Step 4: "Running your products through the gauntlet. This is going to be eye-opening."

Step 1: Connect Your Store

Route: `/onboarding/connect`

Purpose: Import the user's product catalog from their e-commerce platform.

UI Layout

- Headline: "Connect your store"
- Subheadline: "We'll import your product catalog so we can test how AI agents see your products."
- Grid of platform cards (logo + name + "Connect" button):
  - **Shopify** (primary, featured) — OAuth flow
  - **WooCommerce** — API key entry (store URL + consumer key + consumer secret)
  - **BigCommerce** — OAuth flow
  - **Amazon Seller Central** — SP-API OAuth flow

- **Custom Feed** — URL input for XML/JSON product feed
- **CSV Upload** — Drag-and-drop file upload zone

Shopify Connection Flow

1. User clicks "Connect Shopify"
2. Redirect to Shopify OAuth consent screen
3. On approval, receive access token
4. Begin product sync (background job):
   - Fetch all products via Shopify Admin API (GraphQL, paginated)
   - For each product: extract title, description (HTML → plain text), price, compare_at_price, images, variants, tags, product_type, vendor, metafields
   - Store in `products` table with `source = 'shopify'` and `external_id`
   - Extract structured attributes from description using AI (background job)
5. Show progress: "Importing X products..." with a progress bar
6. On completion: Show summary card: "✓ 247 products imported from your-store.myshopify.com"
7. Auto-advance to Step 2

CSV Upload Flow

1. User drops CSV file or clicks to browse
2. Validate CSV structure:
   - Required columns: `title`, `price`
   - Optional columns: `description`, `image_url`, `category`, `brand`, `sku`, `barcode`, `attributes` (JSON string)
3. Show preview table of first 5 rows
4. Column mapping UI if headers don't match expected names
5. "Import" button → background job processes CSV
6. Validation errors shown inline (e.g., "Row 12: missing title")
7. Summary on completion

Error States

- OAuth failure: "We couldn't connect to Shopify. Please try again." + retry button
- No products found: "Your store doesn't have any published products. Make sure you have at least one active product."

- API rate limit: "We're importing your products but Shopify is asking us to slow down. This might take a few extra minutes."
- CSV format error: Show specific column/row errors with suggestions

---

Step 2: Select Products for Initial Scan

Route: `/onboarding/select-products`

Purpose: Choose which products to include in the first simulation run.

UI Layout

- Headline: "Choose products to scan"
- Subheadline: "Select the products you want to test against AI shopping agents. [Tier limit] We recommend starting with your best sellers."
- **Tier badge**: Shows limit — "Free: 10 products" / "Compete: 100 products" / "Dominate: 1,000 products"
- **Search bar**: Filter products by title
- **Category filter**: Dropdown of auto-detected categories
- **Sort**: By title (A-Z), price (high-low, low-high), date added
- **"Select All" / "Deselect All"** toggle (respects tier limit)
- **Smart suggestion banner**: " ⚡ Quick pick: We've pre-selected your 10 most-viewed products" (if Shopify analytics data is available)

Product Table

Column	Content
Checkbox	Select/deselect
Image	Product thumbnail (60x60, rounded)
Title	Product title (truncated at 60 chars)
Price	Formatted price with currency
Category	Auto-detected or from platform
Variants	Count of variants
Data Quality	Mini indicator: 🟢 Good / 🟡 Partial / 🔴 Poor (based on attribute completeness)

Tier Gating

- If user is on free tier and selects > 10 products:
  - Dim additional checkboxes
  - Show inline upgrade nudge: "Upgrade to Compete to scan up to 100 products → See plans"
  - The nudge links to a plan comparison modal, not a new page

"Skip for now" Behavior

- Auto-selects top 10 products by: (1) Shopify analytics popularity if available, (2) most recently updated, (3) alphabetical
- Shows toast: "We've selected 10 products for you. You can change this later."

Step 3: Define Competitor Set

Route: `/onboarding/competitors`

Purpose: Identify which competitor products to benchmark against in simulations.

UI Layout

- Headline: "Who are your competitors?"
- Subheadline: "We'll test your products head-to-head against these competitors in AI agent simulations."

Three Methods of Adding Competitors

Method A: Auto-Detect (Recommended)

- Button: "🔍 Auto-detect competitors"
- Uses selected products' categories and price ranges to find competitor products via:
  - Google Shopping data (product category + price range query)
  - Amazon product matching (similar ASIN lookup)
  - AI-powered semantic matching (product title + description similarity)
- Shows results in a modal: "We found 24 competitor products across 6 brands"
- User can check/uncheck individual competitor products
- Grouped by brand with expand/collapse

Method B: Manual URL Input

- Text area: "Paste competitor product URLs (one per line)"
- Supports: Amazon product pages, Shopify store product pages, any e-commerce product URL
- On paste: background job scrapes product data (title, price, description, specs, images)
- Shows extracted data for confirmation before adding

Method C: Brand Search

- Search field: "Search by brand or category"
- Typeahead results from a database of known e-commerce brands
- "Find competitors in [Wireless Earbuds under $200]" → returns relevant brands + products

Competitor Display

- Card grid showing each competitor product:
  - Product image (if scraped)
  - Product title
  - Price
  - Source (Amazon, Direct, etc.)
  - "Remove" button (x icon)
- Grouped by brand, collapsible
- Counter: "12 competitor products added across 4 brands"

Matching Rules

- Each selected product should have 3–10 competitor products
- If a product has 0 competitors, show warning: "⚠️ No competitors for [Product Name]. Agent simulations require at least 1 competitor."
- Auto-match: Group competitors by product category (e.g., all wireless earbuds compete against each other)

"Skip for now" Behavior

- Run auto-detect with default settings
- Select top 3 competitor products per user product by relevance score
- Show toast: "We've auto-matched competitors for your products. You can refine this later."

Step 4: Run First Simulation

Route: `/onboarding/first-scan`

Purpose: Execute the initial AI agent simulation and deliver the "aha moment."

UI Layout — Pre-Simulation

- Headline: "Ready to see how AI agents judge your products?"
- Summary card showing:
  - Products selected: X
  - Competitors matched: Y
  - Models to test: GPT-4, Claude, Gemini
  - Estimated time: ~Z seconds
- Big CTA button: "🚀 Run Simulation"

UI Layout — During Simulation

Full-screen simulation visualization:

- **Central animation**: An animated visualization of AI agents "shopping" — could be:
  - A terminal-style display showing agent reasoning in real-time (typewriter effect)
  - Animated cards flipping between your product and competitor products
  - A race/progress visualization with each AI model as a "lane"
- **Progress section**:
  - Overall progress bar: "Simulating product 3 of 10..."
  - Per-model status indicators:
    - GPT-4: ✅ Complete | 🔄 Running | ⏳ Queued
    - Claude: ✅ Complete | 🔄 Running | ⏳ Queued
    - Gemini: ✅ Complete | 🔄 Running | ⏳ Queued
- **Live feed** (scrolling log):

```
[GPT-4] Evaluating "Pro Earbuds" vs "SoundMax X3"...
[GPT-4] Checking battery_life: Pro Earbuds = null ❌ | SoundMax = "8 hours" ✓
[GPT-4] Result: SoundMax X3 selected (missing specs)
[Claude] Evaluating "Pro Earbuds" vs "SoundMax X3"...
```

- **Real-time counters** updating as results come in:

- Wins: X
- Losses: Y
- Win Rate: Z%

Technical Implementation

1. User clicks "Run Simulation"
2. Frontend calls `POST /api/simulations/run` with product IDs + competitor IDs
3. Backend creates simulation batch record, queues individual simulation jobs to BullMQ
4. Each job:
   - Constructs a purchase-intent prompt for the specific AI model
   - Sends product data (your product + competitor products) in structured format
   - Parses the model's selection and reasoning
   - Stores result in `simulation_results` table
5. Frontend subscribes to Supabase Realtime (or SSE/WebSocket) for live updates
6. As each simulation completes, UI updates in real-time
7. When all simulations complete → show results summary → auto-redirect to dashboard

UI Layout — Simulation Complete

- **Big number**: "Your Overall Win Rate: 34%"
- **Color-coded**: Red (< 30%), Yellow (30-60%), Green (> 60%)
- **Emotional framing**:
  - < 30%: "AI agents are choosing your competitors 66% of the time. Let's fix that."
  - 30-60%: "You're competitive, but there's room to win more. Let's see where."
  - > 60%: "You're ahead of most competitors. Let's make sure you stay there."
- **Mini breakdown**: Per-model win rates in a horizontal bar chart
- **Top finding**: "Your biggest opportunity: [Product Name] loses on [attribute] across all models."
- **CTA**: "Go to Dashboard →" (primary) / "Share Results" (secondary — generates a shareable image/link)

Post-Simulation Actions

1. Set `onboarding_status = 'complete'` on organization
2. Set `onboarding_completed_at` timestamp
3. Fire `onboarding_completed` analytics event with simulation results summary
4. Queue onboarding completion email (include win rate summary, link to dashboard)

5. If win rate < 40%, queue "quick win" email for 24 hours later with top 3 optimization suggestions

---

5. Dashboard — Global Layout & Navigation

Route: `/dashboard/*`

Layout Structure



Topbar (64px height)

- **Left**: écentric logo (links to dashboard home)
- **Center**: Global search bar — "Search products, competitors, optimizations..." (Cmd+K shortcut)
- **Right**:
  - Notification bell (with unread count badge)
  - Plan badge: "Scan" / "Compete" / "Dominate" / "Enterprise" (click opens billing)
  - User avatar dropdown:
    - User name + email
    - Organization name
    - "Settings"
    - "Billing"
    - "Help & Support"
    - "Sign Out"

Sidebar Navigation (240px width, collapsible to 64px icon-only)

Icon	Label	Route	Badge
📊	Overview	`/dashboard`	—
📦	Products	`/dashboard/products`	Product count
🎯	Competitors	`/dashboard/competitors`	—
⚡	Optimize	`/dashboard/optimize`	Pending count
🔗	UCP & Data	`/dashboard/ucp`	Completeness %
📈	Reports	`/dashboard/reports`	—
⚙️	Settings	`/dashboard/settings`	—

Bottom of sidebar:

- "Run New Simulation" button (prominent blue)
- Current simulation status if one is running: "Simulation in progress... 67%"
- Upgrade CTA for free tier: "Unlock daily simulations → Upgrade"

Responsive Behavior

- **Desktop** (≥1280px): Full sidebar + content
- **Tablet** (768–1279px): Collapsed icon sidebar + content
- **Mobile** (<768px): No sidebar, hamburger menu in topbar, bottom tab navigation for primary routes

---

6. Dashboard — Home / Overview

Route: `/dashboard`

The dashboard home is the command center. It prioritizes actionable insights over vanity metrics.

Top-Level Summary Cards (4-card grid)

Card	Content	Behavior
Overall Win Rate	Large percentage (e.g., "34%") + trend arrow (↑ +3% vs. last week) + mini sparkline (last 30 days)	Click → /dashboard/products filtered by lowest win rate
Products Scanned	"47 / 100" with progress ring showing utilization vs. tier limit	Click → /dashboard/products
Top Performer	Product name + win rate + thumbnail	Click → product detail view
Biggest Opportunity	Product name + current win rate + estimated potential	Click → product detail view

Win Rate by Model (Horizontal Bar Chart)

- One bar per model: GPT-4, Claude, Gemini, Perplexity (if Dominate+)
- Each bar shows win rate percentage
- Color: Blue gradient (darker = higher win rate)
- Tooltip on hover: "GPT-4: 41% win rate across 47 products (192 simulations)"
- Time range selector: Last 7 days / 30 days / 90 days / All time

Win Rate Trend (Line Chart)

- X-axis: Date (daily resolution for ≤30 days, weekly for >30 days)
- Y-axis: Win rate percentage (0–100%)
- Lines: One per model (toggleable) + aggregate line
- Annotations: Markers where optimizations were published (hover shows optimization details)
- Annotations: Markers where model drift was detected

Product Win Rate Table (Top 10, expandable)

Column	Content	Sortable
Product	Thumbnail + title	Yes (A-Z)
Overall Win Rate	Percentage + trend indicator	Yes
GPT-4	Percentage	Yes
Claude	Percentage	Yes
Gemini	Percentage	Yes
Top Competitor	Competitor name + their win rate	No
Gap	Difference between you and top competitor (+ green / - red)	Yes
Status	Badge: "Optimized" / "Needs Review" / "Unoptimized"	Yes
Action	"Optimize →" button (for unoptimized) / "View" (for optimized)	—

- "View All Products →" link at bottom → `/dashboard/products`
- Bulk action bar appears when checkboxes are selected: "Optimize Selected (X)" / "Export Selected"

Recent Activity Feed (Right sidebar or below table)

- Scrollable timeline of recent events:
  - "✅ Optimization published for Pro Earbuds — win rate +12%"
  - "🔄 Daily simulation completed — 3 products improved"
  - "⚠️ Model drift detected on Claude — 7 products affected"
  - "🆕 New competitor detected: SoundMax X3 Pro"
- Each item is clickable → navigates to relevant detail view
- "View All Activity →" link

Quick Actions Bar

- "Run Simulation Now" — triggers immediate simulation for all products
- "View Pending Optimizations (X)" — if there are unapproved optimizations
- "Download Report" — exports current dashboard state as PDF

---

7. Product Catalog

Route: `/dashboard/products`

The central catalog view for all imported products.

Toolbar

- **Search**: Filter by product title (debounced, 300ms)
- **Filters** (collapsible filter bar):
  - Category: multi-select dropdown
  - Win Rate Range: slider (0–100%)
  - Model: GPT-4, Claude, Gemini (filter by specific model performance)
  - Status: Optimized / Needs Review / Unoptimized
  - Store: dropdown (if multiple stores connected)
- **Sort**: Win Rate (ascending/descending), Title (A-Z), Price, Last Simulated
- **View toggle**: Table view / Card grid view
- **Bulk actions** (when items selected):
  - "Run Simulation" — queue simulation for selected products
  - "Optimize All" — generate optimization suggestions for selected
  - "Export" — download CSV/PDF of selected products' data

Product Table (Table View)

Column	Content	Width
☐	Checkbox for bulk selection	40px
Image	Product thumbnail (48x48)	56px
Product	Title + subtitle (category / SKU)	flex
Price	Formatted price	100px
Win Rate	Percentage + color-coded bar (red/yellow/green)	120px
GPT-4	Mini percentage	70px
Claude	Mini percentage	70px
Gemini	Mini percentage	70px
Competitor Gap	"+12%" / "-34%" with color	100px
Data Quality	Score out of 100 + progress ring	100px
Last Simulated	Relative time ("2 hours ago")	120px
Actions	"..." menu: View, Optimize, Run Simulation, Remove	48px

- Pagination: 25 / 50 / 100 per page
- Total count: "Showing 1–25 of 247 products"

Product Card (Grid View)

Each card:



"Add Products" Flow

- Button in toolbar: "+ Add Products"
- Opens modal with options:
  - "Sync from store" — pulls any new products from connected store
  - "Upload CSV" — opens CSV upload flow
  - "Add manually" — form to add single product (title, description, price, image, category, attributes)

---

8. Product Detail View

Route: `/dashboard/products/:id`

The deepest and most important view in the application. This is where merchants understand *why* they're winning or losing.

Header Section

- Product image (large, 200x200)
- Product title (editable inline for manual products)
- Price + category + SKU
- Overall win rate: large number + trend
- Store badge (Shopify icon + store name)
- Action buttons: "Run Simulation", "Optimize", "View in Store" (external link)

Tab 1: Win Rate Overview

Win Rate Over Time (Primary Chart)

- Line chart: daily/weekly win rate
- Multiple lines: aggregate + per-model (toggleable)
- Annotations for optimization events and model drift events
- Date range picker: 7d / 30d / 90d / Custom

Per-Model Breakdown Cards

- 3–5 cards (one per model), each showing:
  - Model name + logo
  - Win rate percentage
  - Wins / Losses / Total simulations
  - Trend: ↑ / ↓ / → with delta
  - "View Decision →" link to Agent Decision Replay

Win Rate by Competitor (Horizontal Bar Chart)

- One bar per competitor product
- Shows your win rate against each specific competitor
- Sorted by win rate (lowest first = biggest opportunities)

Tab 2: Agent Decision Replay

This is the "transparency engine" — the feature that builds trust by showing *exactly* how each AI model evaluates and decides.

Replay Viewer UI

For each simulation run, show a terminal-style decision log:



- **Model selector**: Tabs for GPT-4, Claude, Gemini
- **Simulation run selector**: Dropdown of past simulation timestamps
- **Diff view**: "Compare last 2 runs" to see what changed
- **Highlighted losses**: Red background on attributes where you lost
- **Highlighted wins**: Green background on attributes where you won

---

Tab 3: Attribute Gap Analysis

Attribute Comparison Table

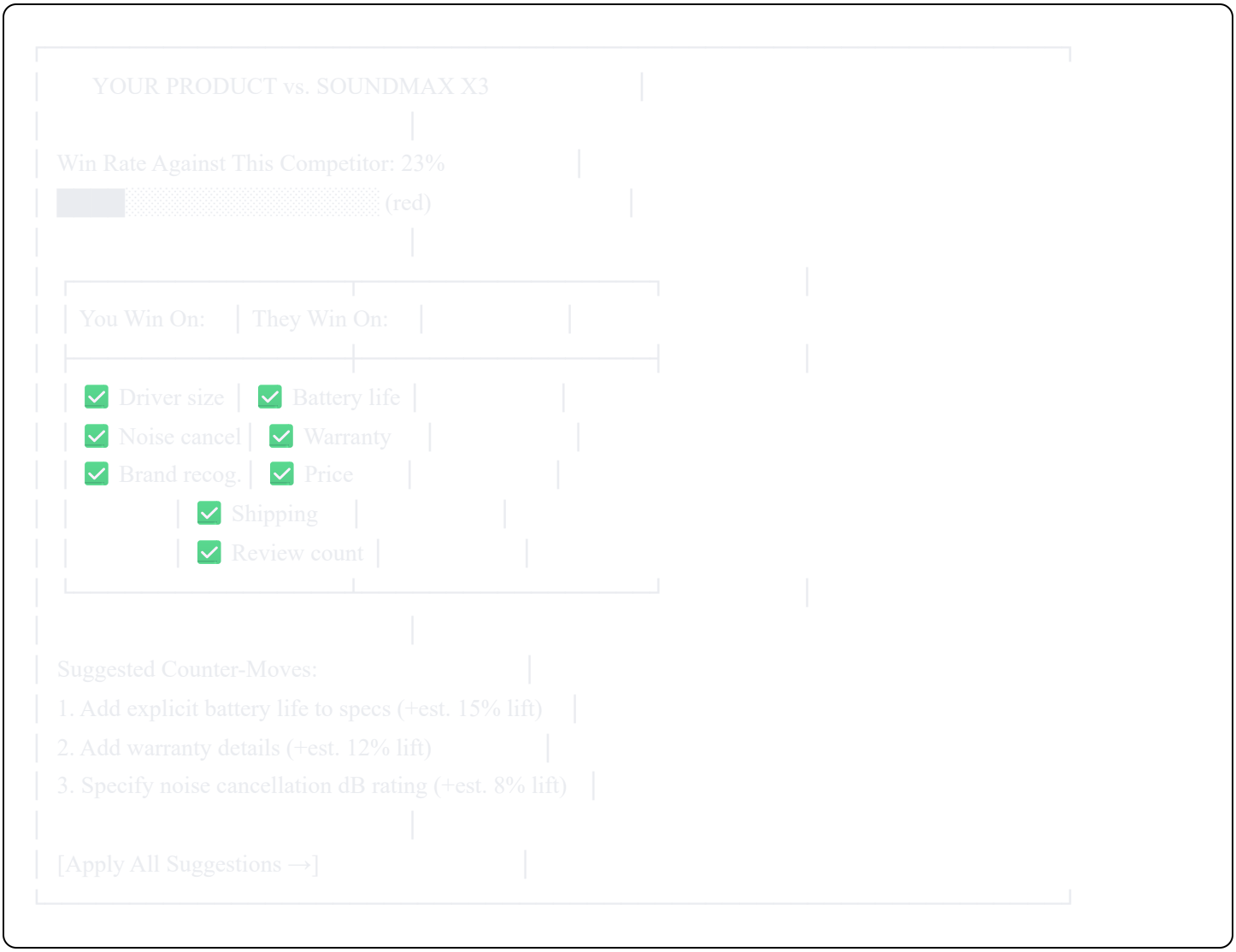
Attribute	Your Product	Top Competitor	Status	Impact
battery_life	❌ Not specified	"8 hours"	🔴 Missing	High — costs 34% of losses
warranty	❌ Not specified	"2 years"	🔴 Missing	High — costs 28% of losses
noise_cancellation	"yes"	"Active, 35dB"	🟡 Weak	Medium — vague vs. specific
driver_size	"12mm"	"10mm"	🟢 Stronger	Low — you win on this
water_resistance	"IPX4"	"IPX4"	🟢 Equal	Neutral
price	"$139"	"$129"	🟡 Higher	Medium — $10 premium
shipping	❌ Not specified	"Free 2-day"	🔴 Missing	Medium

- **Priority ranking**: Sorted by "Impact" — which missing/weak attributes cost the most wins
- **"Fix This" buttons**: For each missing/weak attribute, a button that:
  - Opens inline editor to add/update the attribute
  - Or navigates to the Optimization Panel with this attribute pre-highlighted
- **Data Quality Score**: Calculated score out of 100 based on attribute completeness relative to category norms
- **Category benchmark**: "Products in 'Wireless Earbuds' typically have 18 attributes specified. You have 11."

Tab 4: Competitor Battlecard

Per-competitor detailed breakdown. Dropdown to select specific competitor product.

Battlecard Layout



- Model-specific toggle: See which attributes each model weighs differently
- Historical view: "How has this matchup changed over time?" → line chart

---

Tab 5: Optimization Panel

The action center for this product.

Current vs. Optimized — Side-by-Side Diff View

Field	Current	Optimized	Change
Title	"Pro Earbuds"	"Pro Earbuds — 12mm Drivers, ANC, 10hr Battery, IPX4, 2-Year Warranty"	Added key specs
Description	[current HTML]	[optimized HTML with structured sections]	Restructured for agents
Structured Data	[current JSON-LD or "none"]	[full schema.org Product JSON-LD]	Generated from scratch

Diff Visualization: Side-by-side with additions highlighted in green, removals in red (like a code diff)

Action Buttons:

- "Approve & Publish" — pushes changes to connected store immediately
- "Edit Before Publishing" — opens inline editor on the optimized version
- "Schedule Publish" — date/time picker (Dominate tier)
- "Dismiss" — rejects this optimization, with optional reason
- "Auto-publish" toggle (Dominate tier only) — auto-publishes future optimizations for this product

Optimization History:

- Table of past optimizations with:
  - Date applied
  - What changed (title / description / structured data)
  - Win rate before → win rate after
  - "Rollback" button (reverts to previous version)

---

9. Competitor Intelligence

Route: `/dashboard/competitors`

A dedicated hub for understanding and tracking the competitive landscape.

Competitor Overview

- **Total competitors tracked**: Count
- **Your overall win/loss record**: "Winning against 3 of 8 tracked brands"

- **Most dangerous competitor**: Brand with highest win rate against you

Competitor List Table

Column	Content
Brand	Logo + name
Products Tracked	Count
Your Win Rate	Percentage against this brand
Their Strongest Attribute	What they beat you on most
Your Strongest Attribute	What you beat them on most
Last Updated	When their data was last refreshed
Actions	View Battlecard / Remove / Refresh Data

Attribute Heatmap

A matrix visualization:

- **Rows**: Attributes (battery_life, warranty, price, etc.)
- **Columns**: Competitors
- **Cells**: Color-coded: 🟢 You win / 🟡 Tie / 🔴 They win
- Shows at a glance which attributes are your systematic weaknesses across all competitors

Competitor Alerts

- "🆕 SoundMax just updated their product listing for X3 — added 'Bluetooth 5.3' spec"
- "📈 BassKing Pro's win rate against you increased by 8% this week"
- "🔄 New competitor detected in your category: AudioPeak ZX1"

Add Competitor Flow

Same 3 methods as onboarding (auto-detect, URL paste, brand search) but accessible from within the dashboard.

---

10. Optimization Center

Route: `/dashboard/optimize`

The queue management system for all pending and historical optimizations.

Tabs

Pending (default)

- Queue of optimizations waiting for approval
- Each card shows:
  - Product name + thumbnail
  - Type: Title / Description / Structured Data / All
  - Estimated win rate improvement
  - Models affected
  - "Preview" / "Approve" / "Edit" / "Dismiss" buttons
- Bulk actions: "Approve All" / "Approve Top 10 by Impact"

In Progress

- Optimizations currently being published to stores
- Status: "Publishing to Shopify..." with spinner

Published

- History of all applied optimizations
- Each entry shows:
  - Date published
  - Product name
  - What changed
  - Win rate before → after (with delta)
  - "Rollback" button

Dismissed

- Rejected optimizations with reason (if provided)
- "Reconsider" button to move back to Pending

Auto-Optimization Settings (Dominate Tier)

Accessible from a gear icon in the toolbar:

- **Auto-publish toggle**: Enable/disable fully autonomous publishing
- **Approval threshold**: "Auto-approve if estimated win rate increase > [X]%" (slider, default 10%)
- **Scope**: Apply to all products / specific categories / specific products
- **Frequency**: On every simulation / daily / weekly
- **Safety rails**:
  - "Never auto-publish title changes" toggle
  - "Require manual approval for description changes > 50% different" toggle
  - "Pause auto-publish if win rate drops after optimization" toggle
- **Notification preference**: "Notify me when auto-publishing" toggle

---

11. UCP & Structured Data

Route: `/dashboard/ucp`

Manages the Universal Commerce Protocol merchant profile and per-product structured data.

UCP Merchant Profile

- **Profile completeness**: "Your UCP profile is 68% complete" with progress ring
- **Preview panel**: Shows the JSON output of `/.well-known/ucp` file
- **Editable fields**:
  - Merchant name, description, logo URL
  - Supported payment methods
  - Shipping policies
  - Return policies
  - Contact information
  - Brand values / sustainability info
- **Deploy button**: "Deploy to Store" — installs the `/.well-known/ucp` file:
  - Shopify: Creates a proxy route or uses Shopify Hydrogen
  - WooCommerce: Adds to `.htaccess` or server config
  - Manual: "Copy this JSON and host it at your-domain.com/.well-known/ucp"
- **Validation**: Real-time validation against UCP schema spec with error highlights

Per-Product Structured Data

- Table of all products with structured data status:
  - ✅ Complete JSON-LD generated
  - 🟡 Partial — missing some attributes
  - ❌ No structured data
- **Generate** button per product — AI extracts attributes from images + descriptions
- **Bulk generate** — process all products without structured data
- **Preview**: Show the JSON-LD snippet for any product
- **Install guide**: Code snippet to add to product page templates

Attribute Extraction Engine

For products with incomplete data:

- Upload additional product images
- AI analyzes images for: color, material, dimensions, weight, features visible in photos
- AI parses descriptions for: specifications buried in prose
- Shows extracted attributes for human review before saving
- Confidence scores per extraction: "Battery life: 8 hours (confidence: 92%)"

---

12. Model Drift & Alerts

Accessible from: Dashboard overview alerts + `/dashboard/settings/notifications`

Model drift is when an AI model updates its behavior and changes which products it selects. This is a core differentiator of écentric.

Drift Detection System

- Weekly re-simulation across all products and models (Dominate tier: continuous)
- Compare current simulation results to previous results
- Flag products where win rate changed by ≥5 percentage points
- Identify which model changed and which attributes are now weighted differently

Drift Alert UI

When drift is detected, shows in:

1. **Dashboard banner**: "⚠️ Model drift detected: Claude updated its behavior — 7 products affected"
2. **Notification bell**: Unread notification with drift details
3. **Email alert**: Summary of affected products with links

Drift Detail View

- Which model changed
- Date of detection
- Affected products list with:
  - Previous win rate → Current win rate
  - Which attributes are now valued differently
  - Recommended re-optimization actions
- "Re-optimize affected products" — bulk action button
- Historical drift log: shows all past drift events

---

13. Reports & Exports

Route: `/dashboard/reports`

Available Reports

Report	Description	Format	Tier
Win Rate Summary	Overall performance across all products and models	PDF / CSV	All
Product Performance	Per-product detailed report with trends	PDF / CSV	Compete+
Competitor Analysis	Full competitive landscape with battlecards	PDF	Compete+
Optimization Impact	Before/after analysis of all published optimizations	PDF / CSV	Compete+
Category Benchmark	How your category performs across the agent ecosystem	PDF	Dominate+
Model Drift History	Historical log of all model behavior changes	CSV	Dominate+
UCP Compliance	Audit of UCP profile completeness and issues	PDF	Dominate+

Shareable Results Page

- Generates a public URL for a product's win rate report
- Beautiful, branded single-page report designed for social sharing
- Include: Win rate, model breakdown, top competitor, key attributes
- OG image auto-generated for Twitter/LinkedIn preview cards
- Used for viral "Win Rate" sharing strategy
- Optional password protection

Scheduled Reports

- Configure weekly/monthly email reports
- Select which reports to include
- Add team members as recipients

---

14. Settings

Route: `/dashboard/settings`

Subsections

Account (`/settings`)

- User profile: name, email, avatar
- Password change
- Two-factor authentication toggle
- Delete account

Team (`/settings/team`) — Dominate+ tier

- Invite team members by email
- Role management:
  - Owner: full access
  - Admin: full access except billing and delete
  - Editor: can approve/publish optimizations, run simulations
  - Viewer: read-only dashboard access
- Pending invitations list
- Remove team members

Integrations (`/settings/integrations`)

- Connected stores list with:
  - Platform icon + store name
  - Connection status: Connected / Error / Syncing
  - Last sync timestamp
  - "Sync Now" / "Disconnect" / "Reconnect" buttons
- Add new store connection
- API keys section (Enterprise tier):
  - Generate / revoke API keys
  - Usage tracking per key
- Webhook configuration:
  - Events: simulation_complete, optimization_published, drift_detected, competitor_updated
  - URL + secret key + active toggle per webhook

Notifications (`/settings/notifications`)

- Per-channel (email / in-app / webhook) toggles for:
  - Win rate drops (threshold: configurable, default > 5%)
  - Simulation completed
  - Optimization ready for review
  - Optimization auto-published
  - Model drift detected
  - Competitor listing updated
  - New competitor detected
  - Weekly digest summary
- Quiet hours: "Don't send emails between [time] and [time]"
- Digest mode: "Batch notifications into a daily/weekly summary"

Simulation Config (`/settings/simulation-config`)

- Models to include: checkboxes for GPT-4, Claude, Gemini, Perplexity, others
- Simulation frequency:
  - Scan tier: Manual only
  - Compete tier: Daily / Weekly / Manual

    - Dominate tier: Continuous / 6-hourly / Daily / Manual
- Prompt customization (Enterprise):
    - Purchase persona: "Budget-conscious shopper" / "Premium buyer" / "Technical researcher" / Custom
    - Purchase context: "Quick purchase" / "Researched decision" / "Gift purchase" / Custom
- Competitor matching preferences:
    - Auto-match sensitivity: Conservative / Moderate / Aggressive
    - Price range tolerance: ±10% / ±25% / ±50% / Custom

---

15. Billing & Plan Management

Route: `/dashboard/settings/billing`

Plan Comparison

Interactive pricing table showing current plan highlighted, with feature comparison matrix.

Current Plan Card

- Plan name + price
- Current usage: "47 / 100 products used"
- Billing cycle: monthly / annual
- Next billing date
- Payment method on file (last 4 digits of card)

Upgrade Flow

1. User clicks "Upgrade" on a higher tier
2. Show plan comparison modal highlighting new features
3. Stripe Checkout session for payment
4. On success: immediately unlock new tier features
5. Pro-rate current billing period

Downgrade Flow

1. User clicks "Downgrade" or "Cancel"
2. Show what they'll lose (specific features + data)

3. Retention offer: "Stay on Compete and get 20% off for 3 months?"
4. If confirmed: plan changes at end of current billing period
5. If downgrading below current product count: prompt to select which products to keep

Invoice History

- Table of past invoices with download links (PDF)
- Stripe Customer Portal link for advanced billing management

---

16. Notifications System

In-App Notifications

- **Bell icon** in topbar with unread count badge
- **Notification dropdown** (max-height scrollable):
  - Each notification has: icon, title, description, timestamp, read/unread indicator
  - Click navigates to relevant page
  - "Mark all as read" button
  - "View all" → full notification center page

Notification Types

Type	Icon	Title Pattern	Priority
Win Rate Drop	📉	"Pro Earbuds win rate dropped to 18%"	High
Simulation Complete	✅	"Daily simulation completed — 3 products improved"	Medium
Optimization Ready	⚡	"5 new optimizations ready for review"	Medium
Auto-Published	🚀	"Auto-published optimization for Travel Mug"	Low
Model Drift	⚠️	"Claude model drift detected — 7 products affected"	High
Competitor Update	🔄	"SoundMax X3 listing updated — new specs added"	Medium
New Competitor	🆕	"New competitor detected: AudioPeak ZX1"	Low
Plan Limit	🔒	"You've reached your product limit. Upgrade to add more."	Medium
Weekly Digest	📊	"Your weekly agent performance summary is ready"	Low

17. Data Model — Complete Schema

sql

```sql
-- Organizations (multi-tenant)
CREATE TABLE organizations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  slug TEXT UNIQUE NOT NULL,
  plan TEXT NOT NULL DEFAULT 'scan', -- 'scan' | 'compete' | 'dominate' | 'enterprise'
  plan_status TEXT NOT NULL DEFAULT 'active', -- 'active' | 'past_due' | 'canceled'
  stripe_customer_id TEXT,
  stripe_subscription_id TEXT,
  product_limit INTEGER NOT NULL DEFAULT 10,
  onboarding_status TEXT NOT NULL DEFAULT 'pending', -- 'pending' | 'complete'
  onboarding_step TEXT DEFAULT 'connect',
  onboarding_completed_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Users
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  clerk_id TEXT UNIQUE NOT NULL,
  email TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
  avatar_url TEXT,
  role TEXT NOT NULL DEFAULT 'owner', -- 'owner' | 'admin' | 'editor' | 'viewer'
  organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  signup_source TEXT, -- 'product_hunt' | 'twitter' | 'google' | etc.
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Connected Stores
CREATE TABLE stores (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  platform TEXT NOT NULL, -- 'shopify' | 'woocommerce' | 'bigcommerce' | 'amazon' | 'csv' | 'manual'
  store_name TEXT,
  store_url TEXT,
  access_token TEXT, -- encrypted
  refresh_token TEXT, -- encrypted
  sync_status TEXT DEFAULT 'idle', -- 'idle' | 'syncing' | 'error' | 'complete'
  last_synced_at TIMESTAMPTZ,
  product_count INTEGER DEFAULT 0,
```

```
  metadata JSONB DEFAULT '{}', -- platform-specific config
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Products
CREATE TABLE products (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  store_id UUID REFERENCES stores(id) ON DELETE SET NULL,
  external_id TEXT, -- platform product ID
  title TEXT NOT NULL,
  description TEXT,
  description_html TEXT,
  price DECIMAL(10,2),
  compare_at_price DECIMAL(10,2),
  currency TEXT DEFAULT 'USD',
  category TEXT,
  brand TEXT,
  sku TEXT,
  barcode TEXT,
  image_urls TEXT[] DEFAULT '{}',
  tags TEXT[] DEFAULT '{}',
  attributes JSONB DEFAULT '{}', -- key-value structured attributes
  structured_data JSONB, -- generated JSON-LD
  data_quality_score INTEGER, -- 0-100
  is_selected BOOLEAN DEFAULT FALSE, -- selected for simulation
  current_win_rate DECIMAL(5,2), -- cached aggregate
  win_rate_trend DECIMAL(5,2), -- change from previous period
  optimization_status TEXT DEFAULT 'unoptimized', -- 'unoptimized' | 'pending' | 'optimized'
  last_simulated_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Product Variants
CREATE TABLE product_variants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id UUID REFERENCES products(id) ON DELETE CASCADE,
  title TEXT,
  sku TEXT,
  price DECIMAL(10,2),
  compare_at_price DECIMAL(10,2),
  attributes JSONB DEFAULT '{}',
```

```
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Competitors (brands)
CREATE TABLE competitors (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  brand_name TEXT NOT NULL,
  brand_url TEXT,
  logo_url TEXT,
  source TEXT, -- 'auto_detected' | 'manual' | 'search'
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Competitor Products
CREATE TABLE competitor_products (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  competitor_id UUID REFERENCES competitors(id) ON DELETE CASCADE,
  organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT,
  price DECIMAL(10,2),
  currency TEXT DEFAULT 'USD',
  source_url TEXT,
  image_urls TEXT[] DEFAULT '{}',
  attributes JSONB DEFAULT '{}',
  category TEXT,
  last_scraped_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Product-Competitor Matchings
CREATE TABLE product_competitor_matches (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id UUID REFERENCES products(id) ON DELETE CASCADE,
  competitor_product_id UUID REFERENCES competitor_products(id) ON DELETE CASCADE,
  match_type TEXT DEFAULT 'auto', -- 'auto' | 'manual'
  relevance_score DECIMAL(5,2),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(product_id, competitor_product_id)
);
```

```
-- Simulation Batches
CREATE TABLE simulation_batches (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  trigger_type TEXT NOT NULL, -- 'manual' | 'onboarding' | 'scheduled' | 'drift_rerun'
  status TEXT NOT NULL DEFAULT 'queued', -- 'queued' | 'running' | 'completed' | 'failed'
  total_simulations INTEGER DEFAULT 0,
  completed_simulations INTEGER DEFAULT 0,
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Individual Simulation Results
CREATE TABLE simulation_results (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  batch_id UUID REFERENCES simulation_batches(id) ON DELETE CASCADE,
  organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  product_id UUID REFERENCES products(id) ON DELETE CASCADE,
  competitor_product_id UUID REFERENCES competitor_products(id) ON DELETE CASCADE,
  model TEXT NOT NULL, -- 'gpt-4' | 'claude' | 'gemini' | 'perplexity'
  result TEXT NOT NULL, -- 'win' | 'loss'
  selected_product_id UUID, -- which product the agent chose (could be yours or competitor's)
  decision_reasoning TEXT, -- full text reasoning from the model
  attribute_scores JSONB, -- per-attribute evaluation: { "battery_life": { "yours": null, "theirs": "8 hours", "verdict": "loss" } }
  prompt_used TEXT, -- the full prompt sent to the model
  model_response TEXT, -- raw model response
  latency_ms INTEGER, -- response time
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Optimizations
CREATE TABLE optimizations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  product_id UUID REFERENCES products(id) ON DELETE CASCADE,
  optimization_type TEXT NOT NULL, -- 'title' | 'description' | 'structured_data' | 'attributes'
  original_value TEXT,
  optimized_value TEXT,
  diff_summary TEXT, -- human-readable summary of changes
  estimated_win_rate_lift DECIMAL(5,2),
  status TEXT NOT NULL DEFAULT 'pending', -- 'pending' | 'approved' | 'publishing' | 'published' | 'dismissed' | 'rolled_back'
  approved_by UUID REFERENCES users(id),
  approved_at TIMESTAMPTZ,
```

```
  published_at TIMESTAMPTZ,
  dismissed_at TIMESTAMPTZ,
  dismissed_reason TEXT,
  rolled_back_at TIMESTAMPTZ,
  win_rate_before DECIMAL(5,2),
  win_rate_after DECIMAL(5,2), -- measured after publication
  auto_generated BOOLEAN DEFAULT TRUE,
  auto_published BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- UCP Profiles
CREATE TABLE ucp_profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  merchant_name TEXT,
  merchant_description TEXT,
  logo_url TEXT,
  payment_methods TEXT[] DEFAULT '{}',
  shipping_policies JSONB DEFAULT '{}',
  return_policies JSONB DEFAULT '{}',
  contact_info JSONB DEFAULT '{}',
  custom_fields JSONB DEFAULT '{}',
  completeness_score INTEGER DEFAULT 0, -- 0-100
  deployed_at TIMESTAMPTZ,
  ucp_json JSONB, -- the generated /.well-known/ucp content
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Model Drift Events
CREATE TABLE drift_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  model TEXT NOT NULL, -- which model drifted
  detected_at TIMESTAMPTZ DEFAULT NOW(),
  affected_product_count INTEGER,
  avg_win_rate_change DECIMAL(5,2),
  details JSONB, -- per-product impact details
  status TEXT DEFAULT 'detected', -- 'detected' | 'acknowledged' | 're_optimized'
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```sql
-- Notifications
CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  type TEXT NOT NULL, -- 'win_rate_drop' | 'simulation_complete' | 'optimization_ready' | 'drift_detected' | etc.
  title TEXT NOT NULL,
  body TEXT,
  link TEXT, -- internal route to navigate to
  read BOOLEAN DEFAULT FALSE,
  read_at TIMESTAMPTZ,
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Activity Log
CREATE TABLE activity_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  user_id UUID REFERENCES users(id),
  action TEXT NOT NULL, -- 'simulation_started' | 'optimization_published' | 'competitor_added' | etc.
  entity_type TEXT, -- 'product' | 'optimization' | 'competitor' | 'simulation'
  entity_id UUID,
  details JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_products_org ON products(organization_id);
CREATE INDEX idx_products_selected ON products(organization_id, is_selected);
CREATE INDEX idx_simulation_results_product ON simulation_results(product_id, created_at DESC);
CREATE INDEX idx_simulation_results_batch ON simulation_results(batch_id);
CREATE INDEX idx_optimizations_product ON optimizations(product_id, status);
CREATE INDEX idx_optimizations_status ON optimizations(organization_id, status);
CREATE INDEX idx_notifications_user ON notifications(user_id, read, created_at DESC);
CREATE INDEX idx_activity_log_org ON activity_log(organization_id, created_at DESC);
CREATE INDEX idx_competitor_matches ON product_competitor_matches(product_id);
```

---

18. API Routes & Contracts

Authentication (handled by Clerk middleware)

All `/api/*` routes require valid session. Organization context derived from user's `organization_id`.

Core API Routes

```
# Auth & Onboarding
POST   /api/auth/signup                → Create user + org
GET    /api/onboarding/status          → Get current onboarding step
POST   /api/onboarding/complete-step   → Mark step complete, advance

# Stores
POST   /api/stores/connect             → Initiate OAuth or save credentials
POST   /api/stores/:id/sync            → Trigger product sync
DELETE /api/stores/:id                 → Disconnect store
GET    /api/stores                     → List connected stores

# Products
GET    /api/products                   → List products (paginated, filterable)
GET    /api/products/:id               → Get product detail
PUT    /api/products/:id               → Update product
POST   /api/products/import            → Import from CSV/feed
POST   /api/products/select            → Batch select/deselect for simulation
DELETE /api/products/:id               → Remove product
GET    /api/products/:id/simulations   → Get simulation history for product
GET    /api/products/:id/optimizations → Get optimization history for product
GET    /api/products/:id/competitors   → Get matched competitors for product

# Competitors
GET    /api/competitors                → List competitor brands
POST   /api/competitors                → Add competitor (URL, search, or manual)
DELETE /api/competitors/:id            → Remove competitor
POST   /api/competitors/auto-detect    → Run auto-detection
GET    /api/competitors/:id/products   → List competitor's products
POST   /api/competitors/scrape         → Scrape/refresh competitor data

# Simulations
POST   /api/simulations/run            → Start new simulation batch
GET    /api/simulations/:batchId       → Get batch status + results
GET    /api/simulations/:batchId/stream → SSE stream for live updates
GET    /api/simulations/history        → List past simulation batches
```

```
# Optimizations
GET    /api/optimizations                → List pending optimizations
POST   /api/optimizations/generate       → Generate new optimizations for products
PUT    /api/optimizations/:id/approve    → Approve optimization
PUT    /api/optimizations/:id/publish    → Publish to store
PUT    /api/optimizations/:id/dismiss    → Dismiss optimization
PUT    /api/optimizations/:id/rollback   → Rollback published optimization
POST   /api/optimizations/bulk-approve   → Bulk approve multiple

# UCP
GET    /api/ucp/profile                  → Get UCP profile
PUT    /api/ucp/profile                  → Update UCP profile
POST   /api/ucp/deploy                   → Deploy to store
GET    /api/ucp/products/:id/structured-data → Get product JSON-LD
POST   /api/ucp/products/:id/generate    → Generate structured data for product
POST   /api/ucp/products/bulk-generate   → Bulk generate structured data

# Reports
GET    /api/reports/win-rate-summary     → Aggregate win rate data
GET    /api/reports/product-performance  → Per-product performance data
GET    /api/reports/competitor-analysis  → Competitive landscape data
POST   /api/reports/export               → Generate downloadable report
GET    /api/reports/share/:shareId       → Public shareable report

# Dashboard
GET    /api/dashboard/overview           → Aggregate metrics for dashboard home
GET    /api/dashboard/activity           → Recent activity feed
GET    /api/dashboard/alerts             → Active alerts (drift, drops, etc.)

# Notifications
GET    /api/notifications                → List notifications (paginated)
PUT    /api/notifications/:id/read       → Mark as read
PUT    /api/notifications/read-all       → Mark all as read

# Settings
GET    /api/settings/simulation-config   → Get simulation preferences
PUT    /api/settings/simulation-config   → Update simulation preferences
GET    /api/settings/notification-prefs  → Get notification preferences
PUT    /api/settings/notification-prefs  → Update notification preferences
GET    /api/settings/auto-optimization   → Get auto-optimization rules
PUT    /api/settings/auto-optimization   → Update auto-optimization rules

# Billing
GET    /api/billing/current-plan         → Get current plan details
```

```
POST   /api/billing/create-checkout      → Create Stripe checkout session
POST   /api/billing/create-portal        → Create Stripe customer portal session
POST   /api/billing/webhooks             → Stripe webhook handler
```

---

19. Simulation Engine — Internal Specification

Prompt Architecture

Each simulation is a structured prompt sent to an AI model that simulates a purchase-intent shopping decision.

Prompt Template

You are a shopping assistant AI agent helping a consumer make a purchase decision.

The consumer wants: {purchase_query}

You have access to the following product options:

PRODUCT A:
- Title: {product_a_title}
- Price: {product_a_price}
- Description: {product_a_description}
- Attributes: {product_a_attributes_json}

PRODUCT B:
- Title: {product_b_title}
- Price: {product_b_price}
- Description: {product_b_description}
- Attributes: {product_b_attributes_json}

[Additional products C, D... if applicable]

INSTRUCTIONS:
1. Evaluate each product based on the consumer's needs.
2. Compare products on all relevant attributes: specifications, price, warranty, shipping, reviews, brand reputation, and any other factors a rational consumer would consider.
3. Select the SINGLE best product for the consumer.
4. Explain your reasoning step by step, noting which attributes each product wins or loses on.

Respond in the following JSON format:

```
{
  "selected_product": "A" | "B" | "C" ...,
  "reasoning": "detailed step-by-step explanation",
  "attribute_comparison": {
    "attribute_name": {
      "product_a": "value or null",
      "product_b": "value or null",
      "winner": "A" | "B" | "tie",
      "importance": "high" | "medium" | "low"
    }
  },
  "confidence": 0.0 to 1.0
}
```

Purchase Query Generation

For each product category, generate realistic purchase queries:

- "I need the best {category} under ${price_ceiling}"
- "Help me choose between these {category} options"
- "I want {category} with the best {key_attribute} for my budget"
- Persona variants: budget shopper, premium buyer, technical researcher, gift buyer

Simulation Job Flow

1. Job dequeued from BullMQ
2. Construct prompt with product + competitor data
3. Call AI model API with JSON mode enabled
4. Parse response, validate JSON structure
5. Store result in `simulation_results`
6. Update product's cached `current_win_rate`
7. Emit real-time event for frontend updates
8. Check for drift (compare to previous results)
9. If drift detected → create `drift_events` record

Rate Limiting & Cost Management

- Queue priority: onboarding simulations > manual triggers > scheduled runs
- Concurrency limits per model API to stay within rate limits
- Cost tracking per simulation (estimated token usage)
- Monthly cost budget alerts for Enterprise clients

---

20. Auto-Optimization Engine

Optimization Generation Flow

1. After simulation results are stored, identify products with win rate < threshold (default 50%)
2. For each underperforming product, analyze `attribute_scores` across all simulations
3. Identify top 3 gaps causing the most losses
4. Generate optimized content using AI:
    - **Title**: Add missing key specs, restructure for machine readability

- **Description**: Reorganize with structured sections, add missing attributes as explicit bullet points
- **Structured Data**: Generate full schema.org Product JSON-LD with all available attributes

5. Calculate estimated win rate improvement based on gap analysis
6. Create `optimizations` records with status `pending`
7. Notify user of pending optimizations

Publishing Flow

1. User approves (or auto-approval triggers)
2. Set optimization status to `publishing`
3. Call store API to update product:
   - Shopify: `productUpdate` GraphQL mutation
   - WooCommerce: `PUT /products/{id}` REST endpoint
4. On success: set status to `published`, record `published_at`
5. Schedule re-simulation 24 hours after publication to measure impact
6. After re-simulation: record `win_rate_after`

Rollback Flow

1. User clicks "Rollback"
2. Retrieve `original_value` from optimization record
3. Push original content back to store via API
4. Set status to `rolled_back`, record timestamp
5. Schedule re-simulation to confirm rollback

---

21. Tier Gating & Feature Flags

Feature Access Matrix

Feature	Scan (Free)	Compete ($149)	Dominate ($449)	Enterprise
Product limit	10	100	1,000	Custom
Simulation models	3	3	5+	Custom
Simulation frequency	One-time	Daily	Continuous	Custom

Feature	Scan (Free)	Compete ($149)	Dominate ($449)	Enterprise
Win rate dashboard	✅	✅	✅	✅
Attribute gap report	Basic	Full	Full	Full
Agent decision replay	Last run only	Full history	Full history	Full history
Competitor battlecards	❌	✅	✅	✅
Manual optimization suggestions	❌	✅	✅	✅
Auto-optimization	❌	❌	✅	✅
One-click publish	❌	❌	✅	✅
Model drift alerts	❌	❌	✅	✅
UCP profile generation	❌	❌	✅	✅
Structured data generation	❌	Basic	Full	Full
Team management	❌	❌	❌	✅
SSO / SAML	❌	❌	❌	✅
API access	❌	❌	❌	✅
Shareable reports	✅	✅	✅	✅
Export reports	❌	✅	✅	✅
Scheduled reports	❌	❌	✅	✅
Priority support	❌	❌	❌	✅

Gating Implementation

- Use a `checkFeatureAccess(orgId, feature)` utility function
- Returns `{ allowed: boolean, tier_required: string, message: string }`
- When a feature is gated:
  - Show the feature UI grayed/blurred with a lock icon
  - Overlay: "Upgrade to {tier} to unlock {feature}" with CTA button
  - Never hide gated features entirely — users should see what they're missing

---

22. Empty States, Loading States & Error Handling

Empty States

Every page/section has a designed empty state with:

- Illustration or icon
- Headline explaining what will appear here
- Description of how to populate it
- Primary CTA button

Page	Empty State Message	CTA
Products	"No products yet. Connect your store or upload a CSV to get started."	"Connect Store" / "Upload CSV"
Competitors	"No competitors tracked yet. Add competitors to see how you stack up."	"Add Competitors"
Optimizations	"No pending optimizations. Run a simulation to generate optimization suggestions."	"Run Simulation"
Simulation Results	"No simulation data yet. Run your first simulation to see how AI agents evaluate your products."	"Run Simulation"
UCP	"Your UCP profile hasn't been set up yet. Generate your merchant profile to help AI agents find you."	"Generate Profile"
Notifications	"You're all caught up! No new notifications."	—

Loading States

- **Page loads**: Full-page skeleton loader matching the layout structure
- **Table loads**: Skeleton rows (8 rows) with shimmer animation
- **Chart loads**: Skeleton chart area with animated gradient
- **Action buttons**: Disabled state with spinner while processing
- **Simulation progress**: Dedicated progress UI (see onboarding Step 4)
- **Long operations** (> 5 seconds): Show progress percentage or step count

Error Handling

- **API errors**: Toast notification with error message + retry button
- **Network offline**: Persistent banner: "You're offline. Changes will sync when you're back online."
- **Store sync failures**: Inline error on store card with "Retry" / "Reconnect" options
- **Simulation failures**: Per-model error handling — if one model fails, continue with others, show partial results with note
- **Stripe errors**: Redirect to billing page with error explanation
- **404 pages**: Custom "Product not found" page with search + navigation back to catalog
- **Rate limits**: "We're processing a lot of data right now. Your results will be ready shortly."
- **Optimistic updates**: For approvals and dismissals, update UI immediately, revert on failure

---

23. Accessibility & Responsive Design

Accessibility (WCAG 2.1 AA)

- All interactive elements are keyboard navigable
- Focus indicators on all focusable elements
- ARIA labels on icons, charts, and complex UI components
- Color is never the sole indicator of state (always paired with icons or text)
- Minimum contrast ratio 4.5:1 for text
- Screen reader announcements for live updates (simulation progress, notifications)
- Skip navigation link
- Proper heading hierarchy (h1 → h6)

Responsive Breakpoints

Breakpoint	Layout Changes
≥1536px (2XL)	Full layout, extra chart detail
≥1280px (XL)	Full sidebar + content
≥1024px (LG)	Collapsed sidebar + content
≥768px (MD)	No sidebar, hamburger menu, stacked cards
<768px (SM)	Mobile layout, bottom tab nav, single-column content

Mobile-Specific Adaptations

- Win rate table → horizontal scroll or card stack
- Charts → simplified with fewer data points
- Decision replay → accordion view instead of terminal
- Side-by-side diffs → stacked (current above, optimized below)
- Battlecards → swipeable cards

---

24. Analytics & Tracking Events

Key Events (PostHog / Segment)

Event	Properties	Trigger
`user_signed_up`	source, role, signup_method	Account creation
`store_connected`	platform, product_count	Store OAuth complete
`onboarding_step_completed`	step_number, step_name, duration_seconds	Each onboarding step
`onboarding_completed`	total_duration, products_selected, competitors_matched	Final step done
`simulation_started`	product_count, model_count, trigger_type	Simulation initiated
`simulation_completed`	product_count, avg_win_rate, duration_seconds	All results in
`product_viewed`	product_id, win_rate	Product detail opened
`decision_replay_viewed`	product_id, model	Decision replay tab viewed
`optimization_generated`	product_id, type, estimated_lift	Auto-optimization created
`optimization_approved`	product_id, type	User approves
`optimization_published`	product_id, type, auto_published	Pushed to store
`optimization_dismissed`	product_id, type, reason	User dismisses
`optimization_rolled_back`	product_id, type	Rollback initiated
`competitor_added`	method, competitor_count	Competitor added

Event	Properties	Trigger
`report_exported`	report_type, format	Report downloaded
`report_shared`	report_type	Share link generated
`upgrade_cta_clicked`	current_plan, target_plan, trigger_location	Upgrade button clicked
`plan_upgraded`	from_plan, to_plan	Subscription upgraded
`plan_downgraded`	from_plan, to_plan	Subscription downgraded
`feature_gated`	feature_name, current_plan, required_plan	User hits paywall
`ucp_profile_deployed`	completeness_score	UCP deployed to store

Conversion Funnels to Track

1. **Signup → First Simulation**: signup → connect_store → select_products → add_competitors → run_simulation
2. **Free → Paid**: feature_gated → upgrade_cta_clicked → plan_upgraded
3. **Optimization Impact**: optimization_generated → approved → published → measured_impact
4. **Retention Loop**: simulation_completed → win_rate_viewed → optimization_published → re_simulation → improvement_measured

---

25. Product Hunt Launch Considerations

Launch-Day Features

These features specifically support a Product Hunt #1 launch:

1. **Instant Free Scan**: Zero-friction entry. Paste a Shopify URL, get win rate in < 2 minutes. No credit card.
2. **Shareable Win Rate Page**: Beautiful, branded results page with OG image for social sharing. Every scan generates a shareable link.
3. **Social Proof Counters**: "X,XXX stores scanned today" displayed on landing and during onboarding.
4. **Launch Day Banner**: "We're live on Product Hunt! 🚀" with upvote link — shown in-app to existing users.
5. **Early Adopter Pricing**: "Product Hunt Special: 50% off first 3 months" with countdown timer.
6. **Quick Win Email**: 24 hours after scan, send an email with top 3 specific optimizations the user can make (drives return visits and upgrades).

7. **Category Reports**: Pre-built reports for popular categories (wireless earbuds, protein powder, phone cases, etc.) that can be shared as content marketing alongside the launch.
8. **Embed Widget**: A small embeddable badge showing "Agent Win Rate: XX%" that merchants can put on their site — drives viral awareness.

First 5 Minutes Experience (Critical for PH)

The experience from "Get Started" to "seeing your win rate" must be polished and fast:

```
0:00 — Click "Get Started"
0:15 — Sign up (Shopify OAuth preferred = fewest clicks)
0:30 — Store connected, products importing
0:45 — Products imported, auto-selected top 10
1:00 — Competitors auto-detected
1:15 — Click "Run Simulation"
1:45 — Watch live simulation with real-time results
2:00 — See overall win rate + top finding
2:05 — "Share Your Results" + "Go to Dashboard"
```

Post-Launch Growth Loops

1. **Win Rate Virality**: Users share win rate scores on Twitter/LinkedIn (built into the product as a primary CTA)
2. **Competitive FOMO**: "Your competitor [brand] is already using écentric" notifications when a competitor signs up (opt-in, privacy-respecting)
3. **Agency Multiplier**: Each agency client = 10–100 brands using écentric
4. **Data Journalism**: Monthly category reports published as blog content ("Which wireless earbuds do AI agents prefer?")
5. **Shopify App Store SEO**: Rank for "AI optimization", "agent commerce", "product listing optimization"

Appendix A: Color System & Design Tokens

Based on the écentric brand from the pitch deck (blue + dark theme):

Token	Value	Usage
`--brand-primary`	#2563EB (Blue 600)	Primary buttons, links, active states
`--brand-primary-hover`	#1D4ED8 (Blue 700)	Hover states
`--brand-accent`	#3B82F6 (Blue 500)	Highlights, badges
`--bg-primary`	#FFFFFF	Main content background
`--bg-secondary`	#F8FAFC (Slate 50)	Sidebar, cards
`--bg-dark`	#0F172A (Slate 900)	Dark mode / dark sections
`--text-primary`	#0F172A (Slate 900)	Primary text
`--text-secondary`	#64748B (Slate 500)	Secondary text
`--win-high`	#22C55E (Green 500)	Win rate > 60%
`--win-medium`	#EAB308 (Yellow 500)	Win rate 30–60%
`--win-low`	#EF4444 (Red 500)	Win rate < 30%
`--border`	#E2E8F0 (Slate 200)	Borders, dividers

Appendix B: Key Terminology

Term	Definition
Win Rate	Percentage of simulations where your product was selected by an AI agent over competitors
Agent Commerce	E-commerce transactions mediated by AI shopping agents on behalf of consumers
Simulation	A controlled test where an AI model is given purchase-intent query and product data, then asked to select the best product
Model Drift	When an AI model updates its weights/behavior and changes its product selection patterns
Battlecard	Per-competitor analysis showing exactly which attributes you win/lose on
UCP	Universal Commerce Protocol — Google & Shopify's open standard for agent-to-merchant communication

Term	Definition
Attribute Gap	A product attribute that your competitor specifies but you don't, causing agent preference for the competitor
Decision Replay	The full reasoning trace from an AI model explaining why it chose a specific product
Auto-Write Agent	The AI system that generates optimized product titles, descriptions, and structured data
Data Quality Score	A 0–100 score measuring how complete your product's structured attributes are relative to category norms

This document should be the primary reference for any developer or AI coding agent building the écentric post-landing-page application. Every route, component, data model, and interaction described here represents the target product specification.